



Neuro-symbolic approaches to the SAT problem

Pattern Recognition &
Reinforcement Learning
(survey with coding project)

Donato Meoli

The SAT problem

- ▶ **SAT**: is there an assignment satisfying a Boolean (CNF) formula? First problem proven NP-complete; core of verification, planning, synthesis.
- ▶ Modern solvers are **CDCL**: decide a variable, unit-propagate, on a conflict learn a clause and backtrack.
- ▶ The **branching heuristic** (e.g. VSIDS) picks the decision variable — and makes poor choices during its warm-up phase.
- ▶ *This project*: learn the branching heuristic with two neuro-symbolic methods, modernise them to PyTorch, reproduce, and study **where graph attention helps**.

Contributions

- ▶ Port both methods from **TensorFlow 1.x + GSL** to one self-contained **PyTorch / PyTorch-Geometric** stack (latest torch, numpy $\geq$ 2, gymnasium; no torch-scatter/sparse, no GSL).
- ▶ **Reproduce the published Graph-Q-SAT results exactly** (semantic non-regression oracle).
- ▶ **GAT-Q-SAT**: an attention-augmented variant, with a clean attention-on/off study showing *where* attention helps.

- ▶ NeuroSAT
- ▶ Background
 - ▶ Message Passing NNs (MPNNs)
- ▶ Model
- ▶ Training
- ▶ Results

(ISPR – survey)

Selsam et al., *Learning a SAT Solver from Single-Bit Supervision*.

NeuroSAT

NeuroSAT (survey)

- ▶ **Goal:** can a neural network learn to solve SAT?
- ▶ An MPNN trained *only* as a SAT/UNSAT classifier (one-bit supervision) nonetheless learns to find satisfying assignments.
- ▶ Introduces the **graph view** of a CNF (variable/clause nodes) that underpins Graph-Q-SAT — our second method.

- ▶ AlphaZeroSAT
- ▶ Background
 - ▶ Monte Carlo Tree Search (MCTS)
- ▶ Model
- ▶ Training
- ▶ Results

(ISPR + RL – survey with coding)

Wang & Rompf, *From Gameplay to Symbolic Reasoning*
... *Alpha(Go) Zero*.

A large, solid brown rectangle occupies the right half of the slide. Centered within this rectangle is the text "AlphaZeroSAT" in a white, sans-serif font. The text is split across two lines: "AlphaZero" on the top line and "SAT" on the bottom line, with a hyphen between "Zero" and "SAT".

AlphaZeroSAT

AlphaZeroSAT — model & training

- ▶ State: CNF as a fixed-size **clauses** $\times$ **variables** matrix; a **CNN** with policy head π and value head $v \in [-1, 1]$.
- ▶ MiniSat extended into a *game* env supporting MCTS **simulations**.
- ▶ Self-play + supervised training with the AlphaZero loss
$$-\sum_a \pi_a^{\text{MCTS}} \log \pi_a + (z - v)^2 + c \|\theta\|^2.$$
- ▶ **This work**: ported to PyTorch; **GSL removed** (Dirichlet noise via C++11 `<random>`); trains end-to-end on uf20-91 (CPU/GPU).
- ▶ *Limitation*: the fixed-size CNN **cannot generalise across sizes** — the motivation for graphs.

- ▶ GraphQSAT
- ▶ Background
 - ▶ Deep Q-Learning (DQN)
 - ▶ Graph Neural Networks
- ▶ Model
- ▶ Graph Attention Networks (GATs)
- ▶ Training
- ▶ Results

(ISPR + RL – survey with extra coding)

Kurin et al., *Can Q-Learning with Graph Networks Learn a Generalizable Branching Heuristic for a SAT Solver?*

GraphQSAT

Graph-Q-SAT & GAT-Q-SAT — model

- ▶ State: **bipartite graph** (variable / clause nodes); edge features = literal polarity. Q -function = **encode–process–decode** graph net; DQN.
- ▶ Action: $\arg \max Q$ over variable nodes. Metric: **MRIR** (MiniSat iters / model iters; > 1 is better than MiniSat).
- ▶ **GAT-Q-SAT (this work)**: inject edge-aware, multi-head GATConv attention into the core block — weight the most relevant neighbouring clauses/variables.
- ▶ Port: dropped torch-scatter/sparse; gym-robust env; version-agnostic checkpoint loading.

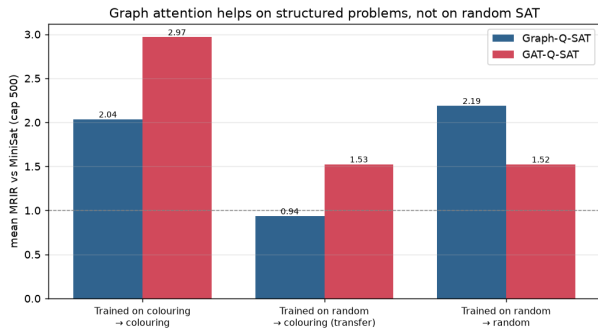
Reproduction (semantic non-regression)

- ▶ Re-ran the *original trained checkpoints* on the modern stack (latest torch / PyG / numpy $\geq$ 2 / gymnasium).
- ▶ MRIR matches the committed logs **exactly**:

Model	MRIR (median)	committed
Graph-Q-SAT	1.833	1.833
GAT-Q-SAT	1.667	1.667

- ▶ Drop-in changes only (scatter, env construction, GATConv key remap) — **semantics preserved**. Figures regenerated from the logs (no Sheets).

Results — the central finding

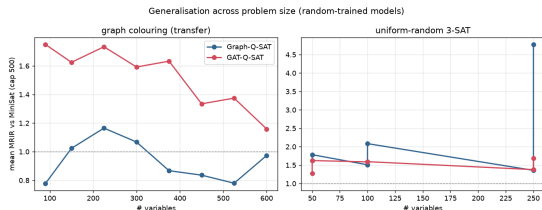


Attention helps on structured problems, not on uniform-random SAT.

Clean attention on/off ablation, matched training sets (mean MRIR, cap 500):

- **Colouring** (in-dist): GAT **2.97** vs 2.04.
- **Transfer** random→colouring: GAT **1.53** vs 0.94 (Graph-Q-SAT < 1, i.e. worse than MiniSat!).
- **Random 3-SAT**: attention *does not* help (1.52 vs 2.19).

Results — attention helps more on *larger* structured problems



Random-trained: MRIR vs problem size (cap 500).

In-distribution graph colouring, MRIR per size:

set	Graph	GAT	Δ
flat30-60	1.83	2.00	+0.17
flat125-301	2.55	3.44	+0.88
flat150-360	1.92	3.76	+1.85
flat200-479	1.35	3.39	+2.04

Plain Graph-Q-SAT **degrades** on large instances; GAT-Q-SAT holds up — the advantage **grows with size**.

Where the field went & conclusions

- ▶ Lineage: NeuroSAT $\rightarrow$ NeuroCore $\rightarrow$ NeuroComb $\rightarrow$ **NeuroBack** (ICLR'24): a Graph *Transformer* with self-attention, *offline* one-shot inference $\Rightarrow$ first wall-clock speed-ups on Kissat.
- ▶ This **validates** the attention direction (GAT-Q-SAT) and addresses Graph-Q-SAT's known limit (online inference too slow).
- ▶ **Our results:** graph attention helps on structured problems (colouring), in-distribution and under transfer, *growing with size*; not on random SAT. The fixed-size CNN (AlphaZeroSAT) cannot generalise.

Code, report and reproducible figures: github.com/dmeoli/NeuroSAT.